

Nap

User Manual

Michael Dungan

Nap Manual

What is Nap?

Nap is a character reverb built on the **Extended Dark Velvet Noise (EDVN)** engine. Its defining feature is that you draw three curves — **Decay**, **Width**, and **Tone** — over a shared tail-position axis (0 → 100 %) that sculpt exactly how the reverb tail's loudness, stereo image, and tonal colour each evolve along their length.

That means you can draw a reverse swell, a plateau that collapses, a tail that starts mono and fans into full stereo at the halfway point, or one that starts bright and dims as it decays — all with the same three curve editors. No commercial reverb offers this level of per-section control.

The DSP is time-domain sparse convolution (no FFT), so Nap reports **zero latency** to the host. Pre-Delay shifts only the wet path; the dry signal stays phase-aligned.

Inspired by the Aalto Acoustics Lab papers on EDVN (Fagerström et al., JAES 2024) and BDVN (DAFx 2024).

Installation

Build from source (requires nightly Rust):

```
cargo nih-plugin bundle nap --release
```

The bundler outputs to target/bundled/. Copy either the .vst3 or .clap file to your plugin directory:

- **Linux:** ~/.vst3/ or ~/.clap/
- **macOS:** ~/Library/Audio/Plug-Ins/VST3/ or ~/Library/Audio/Plug-Ins/CLAP/
- **Windows:** C:\Program Files\Common Files\VST3\ or C:\Program Files\Common Files\CLAP\

Quick Start

1. Insert Nap on a track and play audio through it.
2. Set **Mix** to around 30 % to blend in the tail.
3. The default **Decay** curve falls from full to silence — the tail decays naturally. Try sketching a plateau or a reverse swell.
4. Raise **Width** to spread the tail stereo image; the **Width curve** controls how wide each section of the tail is.
5. The **Tone curve** defaults to bright-at-attack, darkening over time (air absorption). Flatten it or invert it to taste.

6. Adjust **Size** (tail length) and **Density** (pulse count) to change the reverb character. The live tail visualization in the Decay pane updates as you edit.
7. Hit **Seed** / **Regenerate** to re-roll the random velvet pattern while keeping the curve shapes.

Controls

Curve Editors (top three panes)

The three stacked MSEG editors share a **tail-position x-axis** running from 0 % (attack of the tail) to 100 % (end of the tail). All three drive the velvet sequence generation together; any edit triggers an immediate re-generation.

Decay Controls the **energy gain envelope** of the tail. The y-axis maps directly to pulse gain $g(m)$ at each point along the tail.

- **1.0 (top)** → full-amplitude pulses at that tail position.
- **0.0 (bottom)** → silent pulses; the tail is completely dead at that section.
- Draw an **exponential fall** (default) for a natural-sounding room.
- Draw a **gate** (full, then sudden silence) for a gated reverb.
- Draw a **reverse curve** (0 → 1) for a swell-style tail.
- Draw a **plateau** or **multi-bump** shape for rhythmic reverb textures.

The live pulse-field visualization in this pane shows the actual generated sequence: stick height = pulse gain, colour = tone/filter index, horizontal L/R split = width jitter.

Width Controls the **stereo spread** of the tail, section by section. The y-axis maps to the per-pulse maximum jitter $J(m)$ applied to the right-channel pulse locations.

- **0.0 (bottom)** → L and R channels receive identical pulse locations → the tail collapses to mono at that point.
- **1.0 (top)** → maximum jitter (set by the **Width** dial) → widest spread at that point.
- Draw a **flat 0** for a fully mono tail; draw a **flat 1** for uniform stereo spread.
- Draw a **ramp** to have the image open up progressively along the tail.
- At Width dial = 0 ms, the curve has no effect (mono throughout).

The width mechanism is grounded in EDVN/BDVN theory: interaural coherence equals the DTFT of the pulse-location jitter distribution. Animating jitter width over the tail animates the stereo image at zero additional compute cost.

Tone Controls the **spectral colour** of each section of the tail. The y-axis selects among the coloration dictionary's ordered filters:

- **0.0 (bottom)** → darkest filter (strong low-pass character).
- **1.0 (top)** → brightest filter (full high-frequency content).
- The default curve starts at 0.85 and falls to 0.25 — mimicking air absorption (bright-to-dark).
- Flatten it to 1.0 for a consistently bright tail; flatten to 0.0 for a deep, dark reverb throughout.
- Draw a **rise** (dark → bright) for an unusual tonal swell along the tail.

The dictionary contains 6 one-pole lowpass filters ordered dark → bright. Filter routing is per-pulse; the Tone curve value at each pulse's tail position selects the target filter index.

Bottom Strip Dials

Size Tail length, ~0.1 – 10 s. Default: 1.5 s.

Sets the total length of the generated velvet tail. Longer sizes produce larger reverb rooms; combined with a falling Decay curve they give a smooth, room-like decay. Combined with a plateau Decay they produce long, dense sustain.

Design-time parameter — non-automatable. Changing Size re-generates the velvet sequence (an O(M) walk on the GUI thread, not the audio thread). Changes take effect immediately on edit.

Density Pulse density, ~500 – 4000 pulses/s. Default: 1500 /s.

Controls how many velvet pulses are spread per second of tail. Higher density → denser, smoother-sounding tail; lower density → sparser, grainier texture (artefact if too low). The optimal range depends on musical material and the Decay curve.

Design-time parameter — non-automatable. Changing Density re-generates the sequence.

Note: CPU cost scales as **Size × Density** (the sparse-convolution is O(M) per sample). Long tails at high density are expensive; the GUI will still respond but DSP load rises.

Width Maximum right-channel jitter, 0 – 30 ms. Default: 8 ms.

The absolute ceiling for the per-pulse L/R decorrelation offset. The **Width curve** scales this dial value per tail position. At 0 ms the tail is always mono, regardless of the Width curve; at 30 ms the maximum spread reaches 30 ms of physical delay between channels.

Design-time parameter — non-automatable. Changing Width re-generates the sequence.

Pre-Delay Wet-path pre-delay, 0 – 200 ms. Default: 0 ms.

Delays the wet (reverberant) signal relative to the dry signal. The dry signal stays time-aligned; only the reverb tail is pushed back. Use this to simulate room distance — longer pre-delays suggest larger spaces.

Automatable, smoothed.

Mix Dry/wet blend, 0 – 100 %. Default: 30 %.

At 0 % the output is fully dry. At 100 % the output is fully wet (just the tail, no direct signal). Typical room reverb usage: 20–40 %.

Automatable, smoothed.

Input Input trim, –24 – +24 dB. Default: 0 dB.

Scales the signal fed into the reverb engine before velvet convolution. Affects both L and R channels equally. Does not affect the dry path.

Automatable, smoothed.

Output Output trim, –24 – +24 dB. Default: 0 dB.

Scales the final mixed output (after dry/wet blend). Use this to compensate for level changes from Mix or to integrate Nap into a gain-staged mix chain.

Automatable, smoothed.

Seed Integer seed, 1 – 9999. Default: 1.

Selects the random velvet pattern. The generated pulse locations and signs are fully deterministic from the seed and all other design-time parameters. Changing Seed re-rolls the pattern while preserving the curve shapes — the envelope, stereo image, and tonal trajectory all stay the same, but the specific pulse positions shift.

Use this when the current pattern has an audible irregularity or comb artefact. A few seed values usually yield a smooth, natural-sounding tail for any given set of curves.

Design-time parameter — non-automatable. There is a **Regenerate** button beside the dial that re-generates with the current seed (useful when a preset was loaded and the sequence needs to be rebuilt for the current sample rate).

How It Works

EDVN Velvet Noise

Velvet noise is a sparse random impulse sequence with exactly one non-zero pulse per fixed grid cell, with a random sign and location within that cell. A velvet-noise reverberator convolves the input with this sparse sequence, producing a reverberation tail whose density is controlled by the grid spacing.

Extended Dark Velvet Noise (EDVN) extends this with a bank of parallel coloration filters (an all-pole “dictionary”) whose job is to give the tail a realistic spectral character — the “dark” quality of real room tails where high frequencies decay faster than low. Each pulse is routed to one dictionary filter; the routing follows the drawn Tone curve, so tonal evolution is under direct user control.

Decay and Coloration Decoupling

A key insight from the EDVN paper (Fagerström et al., JAES 72(6), 2024) is that decay shape and spectral colouring are **mathematically independent**. The energy envelope $g(m)$ applied to each pulse controls level; the filter routing controls colour; they don’t interact. This is why Nap can offer three independently drawn curves — changing Decay doesn’t change Tone, and vice versa.

Width via Pulse-Location Jitter

From the BDVN paper (DAFx 2024), interaural coherence equals the DTFT of the pulse-location jitter probability distribution. A per-pulse right-channel offset $\Delta(m)$ drawn from a distribution of half-width $J(m)$ gives a frequency-dependent coherence curve where wider J produces lower coherence (less correlation between L and R). Making $J(m)$ time-varying — the Width curve — animates the stereo image across the tail at zero added compute cost.

Sparse Convolution (audio thread)

For each output sample:

1. The input sample is written into a pre-allocated ring buffer.
2. Each active velvet pulse tap reads from the ring buffer at its stored offset, scales by its coefficient, and accumulates into one of Q coloration filter inputs.
3. The Q parallel one-pole IIR filters run on their accumulators.
4. Filter outputs are summed → a gentle post-filter LP → a DC blocker.
5. The result passes through the pre-delay line and is blended with the dry signal.

The left and right channels use the same `coeff[]` and `filter_idx[]` arrays but separate pulse location arrays (`location[]` for L, `location_r[]` for R), so the stereo processing is correct with a single generation pass.

Cost is $O(M)$ per sample where $M = \text{Size} \times \text{Density}$. A 2 s tail at 1500 pulses/s = 3000 taps per sample — well within real-time budget on modern hardware.

Sequence Generation (GUI thread)

The velvet sequence is generated on the GUI thread whenever any design-time parameter or drawn curve changes, then published to the audio thread via a lock-free handoff (`SequenceHandoff`, mirroring `miff's KernelHandoff`). The audio thread calls `try_lock` once per block; if a newer sequence exists, it copies it into its local buffer (pre-allocated to `MAX_PULSES` capacity, no heap allocation) and proceeds.

Interaction

Drawing Curves

All three panes use the shared MSEG editor from `tiny-skia-widgets`. The curve runs left-to-right over tail position ($0 \rightarrow 100\%$).

- **Left-click and drag** on empty space to add and move a node.
- **Drag a node** to reposition it.
- **Drag a segment** (between nodes) to adjust tension (positive tension bows upward, negative bows downward).
- **Alt-drag** on a node or segment for stepped (staircase) draw mode.
- **Double-click a node** to delete it.
- **Right-click** on a node for the context menu (reset tension, delete, etc.).

Any edit triggers immediate re-generation of the velvet sequence. The pulse-field visualization in the Decay pane updates on the next paint frame.

Dial Drag and Text Entry

All bottom-strip dials use the shared `param_dial` widget:

- **Drag vertically** to adjust (up = increase).
- **Shift+drag** for fine control (10× slower).
- **Double-click** to reset to default.
- **Right-click** to type an exact value. Press **Enter** to commit, **Escape** to cancel; clicking outside the edit field auto-commits.

Seed and Regenerate

The **Seed** dial selects the integer seed for the velvet pattern. Dragging it re-rolls the pulse locations. The adjacent **Regenerate** button forces a new generation pass with the current seed — useful after loading a preset to rebuild the sequence at the current sample rate, or to confirm the current sequence is up to date.

Resize

Use the `- / +` buttons in the corner, or **Ctrl+=** / **Ctrl+-** on the keyboard. Scale range: 75 % to 300 %. The scale is persisted across host restarts via `EditorState`.

Technical Notes

- **Zero reported latency** — time-domain causal sparse convolution, no FFT, no lookahead. Pre-Delay is a musical wet-path control, not algorithmic latency, and is not reported to the host.
- **No audio-thread allocations** — ring buffers, filter state, and sequence buffers are pre-allocated to maximum capacity; the handoff uses `try_lock` + a bounded copy, never `Vec::new()` or `clone()` in `process()`.
- **No unsafe code** — beyond the standard baseview/raw-window-handle glue inherited from the editor scaffold (where the FFI requires it).
- **CPU rendering** — tiny-skia (software rasterizer) + fontdue (glyph cache) + softbuffer (pixel buffer). No OpenGL context, no GPU driver overhead per instance.
- **Tail reporting** — `process()` returns `ProcessStatus::Tail` while the velvet tail rings out after the input goes silent, so the host knows not to cut processing prematurely.
- **Feature tier** — Nap's $O(M)$ convolver scales with $\text{Size} \times \text{Density}$ and is intentionally not held to the 100+-instance memory target. The DSP is cheap for typical values, but it is a single-instance character reverb, not a metering utility.

Formats

- CLAP
- VST3
- Standalone (JACK or ALSA backend)

License

GPL-3.0-or-later